

Code Bash 2026

Computer Science Competition

Hands-On Programming Set

I. General Notes

1. Do the problems in any order you like. They do not have to be done in order from 1 to 12.
2. All problems have a value of 60 points.
3. There is no extraneous input. All input is exactly as specified in the problem. Unless specified by the problem, integer inputs will not have leading zeros. Unless otherwise specified, your program should read to the end of file.
4. Your program should not print extraneous output. Follow the form exactly as given in the problem.
5. A penalty of 5 points will be assessed each time that an incorrect solution is submitted. This penalty will only be assessed if a solution is ultimately judged as correct.

II. Point Values and Names of Problems

Number	Name
Problem 1	Adam
Problem 2	Ariel
Problem 3	Barbara
Problem 4	Benjamin
Problem 5	Fred
Problem 6	Lockers
Problem 7	Samuel
Problem 8	Coffee Tower
Problem 9	William
Problem 10	Xander
Problem 11	String Construction Pt. 1
Problem 12	String Construction Pt. 2

1. Adam

Program Name: Adam.java

Input File: adam.dat

Adam just got his very first computer, and he could not be more excited to start programming at Code Bash! To celebrate, he wants to draw a picture of his shiny new computer using ASCII art. Adam does not need any input for this – he already knows exactly what his computer looks like. Help Adam print his picture to the screen.

Input:

There is no input for this problem.

Output:

Print the picture shown below exactly as it appears, using single spaces (no tabs), with no blank lines and no trailing spaces. The row of numbers above the picture is only a ruler to help you count columns – it is NOT part of the output, so do not print it. The top line is a single space followed by 20 underscores.

Sample output:

```
1234567890123456789010 ← ruler (do not print)
```

2. Ariel

Program Name: Ariel.java

Input File: ariel.dat

Ariel has been messing around with codes, but she's not very good at them. Her favorite code lately is to send a list of words and numbers, and you need to get the character at the given index of each word to create the message.

Input:

The input file will begin with an integer, n , denoting how many test cases there will be in the file. Each test case will consist of a string s , containing no spaces, followed by a space, followed by an integer i , denoting the index we are looking for in the given word.

Output:

Output, on a single line, all of the characters for Ariel's code.

Sample input:

```
8
copier 0
poolside 2
handed 5
elephant 2
bubble 3
slap 2
tractors 7
hash 3
```

Sample output:

```
codebash
```

3. Barbara

Program Name: Barbara.java

Input File: barbara.dat

Barbara loves saving space. Instead of writing out long words, she “squishes” them by writing each letter followed by the number of times it repeats in a row. For example, she writes the word bee as b1e2, and balloon as b1a1l1o2n1. Her little brother cannot read her squished notes, so help him expand each one back into the original word.

Input:

The first line of the input file contains an integer, *n*, denoting the number of squished strings. Each of the next *n* lines contains one squished string: a series of pairs, where each pair is a single lowercase letter followed by a positive integer telling how many times that letter is repeated. The integer may be more than one digit.

Output:

For each squished string, output the expanded (original) string on its own line.

Sample input:

```
6
b1e2
pl1l1z2a1
cl01m2i1t2e2
slu1c2e1s2
b1r12
h1a1h1a1
```

Sample output:

```
bee
pizza
committee
success
brrrrrrrrrrrrr
haha
```

4. Benjamin

Program Name: Benjamin.java

Input File: benjamin.dat

Benjamin has begun working in a new job at the mall, and has been tasked with sorting inventory. You need to sort based on the following criteria:

First, sort alphabetically by store name (ASCII order).

Second, sort the items by category in the following order: Clothing,Accessory,Electronics,Books,Misc

Third, sort the items by price, highest to lowest

Finally, sort by item name, alphabetically (ASCII order)

Input:

The input file will begin with an integer, n , denoting how many test cases there will be in the file. Each test case will begin with an integer i , on its own line, denoting the number of items to be tested in this test case. The following i lines will each contain an item in the following format: Item_Name Store_Name Category Price

Output:

For each test case, output the names of the items in sorted order.

Sample input:

```
2
3
Blue_Jeans Old_Navy Clothing 18.99
Watch Old_Navy Accessory 25.50
Long_Sleeve_Shirt Old_Navy Clothing 9.99
3
Blue_Jeans Old_Navy Clothing 18.99
Watch Old_Navy Accessory 25.50
Long_Sleeve_Shirt GAP Clothing 9.99
```

Sample output:

```
Blue_Jeans, Long_Sleeve_Shirt, Watch
Long_Sleeve_Shirt, Blue_Jeans, Watch
```

5. Fred

Program Name: Fred.java

Input File: fred.dat

Fred has built a tiny computer! It has 26 registers, named a through z, and every register starts at 0. The computer runs a program made of numbered instructions (the first instruction is line 1). It carries out the instructions in order from top to bottom, except when a jump instruction sends it to a different line, and it stops when it reaches a `HALT` instruction. Help Fred run his programs and report everything they print.

Fred's computer understands these instructions:

<code>SET x n</code>	store the integer n in register x.
<code>ADD x y</code>	add y to register x.
<code>SUB x y</code>	subtract y from register x.
<code>JMP t</code>	jump to line t.
<code>JZ x t</code>	if register x equals 0, jump to line t; otherwise continue.
<code>PRINT x</code>	output the value of register x on its own line.
<code>HALT</code>	stop the program.

In `ADD` and `SUB`, the second operand `y` may be either a register (a single lowercase letter) or an integer.

Input:

The first line of the input file contains an integer, `n`, the number of programs. Each program begins with a line containing an integer, `m`, the number of instructions, followed by `m` lines each containing one instruction. All register values fit in a 64-bit signed integer, and every program is guaranteed to reach a `HALT`.

Output:

For every `PRINT` instruction that is executed, output the printed value on its own line, in the order the instructions run.

Sample input:

```
3
8
SET a 0
SET b 5
JZ b 7
ADD a b
SUB b 1
JMP 3
PRINT a
HALT
9
SET a 6
SET b 7
```

(cont. input from prev. column)

```
SET c 0
JZ b 8
ADD c a
SUB b 1
JMP 4
PRINT c
HALT
6
SET a 3
PRINT a
SUB a 1
JZ a 6
JMP 2
HALT
```

Sample output:

```
15
42
3
2
1
```

6. Lockers

Program Name: Lockers.java

Input File: lockers.dat

Your school has n seniors, each of which has their own locker. For this year's senior prank, the entire class body of n senior students have decided that they will mess with one another's lockers. The prank will go as follows:

- Each of the n seniors are assigned a number starting from 1 and ending at n .
- All n lockers start closed.
- The i^{th} senior will change the state of every i^{th} locker.
 - If the student's number was $i = 1$, then they would open all n lockers.
 - If the student's number was $i = 2$, then they would close every 2nd locker.
 - If the student's number was $i = 3$, then, for every 3rd locker, if it was open, they would close it, and if it was closed, they would open it.
- This process of flipping the state of every i^{th} locker is repeated until all n seniors have had their turn.

Some of the seniors who haven't paid attention in their math classes see this as a potential game of chance, where they can place bets on whether their or someone else's locker will be open or closed after all n seniors have messed with the lockers; however, you're smarter than that, and know that this process is entirely deterministic. Looking to capitalize off of this situation and hopefully earn some money to go towards your college tuition, you've decided to analyze which lockers will and won't be open, so that you know which bets are worth taking, and which ones you know you would lose at.

Input:

The first line of input will consist of two single-space separated integers: n , denoting the number of students and lockers there are, followed by m denoting the number of queries there will be. The following m lines will each contain a single integer q_i , denoting the locker number in question that you wish to know whether will be opened or closed.

Constraints:

$$1 \leq n \leq 10^{12}$$

$$1 \leq m \leq 10^7$$

$$1 \leq i \leq m, 1 \leq q_i \leq n$$

Output

Print a single integer A denoting the number of the m queries where the locker q_i was open.

Sample input:

```
57 10
1
2
3
4
25
26
32
37
49
53
```

Sample output:

```
4
```

Explanation:

The following lockers are closed after all 57 students have completed their turn:

2 3 26 32 37 53

Since there are six lockers closed and ten total queries, the answer is four.

7. Samuel

Program Name: Samuel.java

Input File: samuel.dat

Samuel has begun playing a tile-stacking game where the goal is to create stable towers. A stable tower consists of exactly n tiles, each stacked such that no larger tile is placed on a smaller one. Each tile has an integer size from 1 through m . Samuel would like you to write a program to determine how many distinct tile towers combinations can be created. Two towers are considered distinct if, at any height h (where $1 \leq h \leq n$), they have tiles of different sizes.

Input:

The input file will begin with an integer, i , denoting how many test cases there will be in the file. Each of the following i lines will contain 3 space-separated integers, n , m , and k , denoting the height of the tower, the maximum size of a tile, and the maximum number of times each tile size can be used in the tower, respectively.

Output:

For each test case, output the number of distinct tower combinations that can be formed.

Sample input:

```
3
3 3 1
3 3 2
6 5 2
```

Sample output:

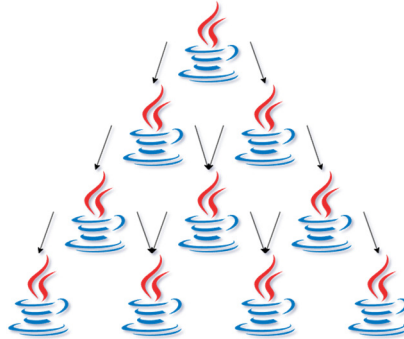
```
1
7
45
```

8. Coffee Tower

Program Name: Coffee.java

Input File: coffee.dat

Your team just finished first place at your local Hackathon but currently, you are far too tired to celebrate as you have spent the last 24 hours working on your award-winning project. Thankfully, the judges have anticipated the sleepiness of the contestants and arranged an infinitely tall tower of coffee mugs, with one coffee mug appearing on the top, being balanced by two coffee mugs below it, all being balanced by three coffee mugs below those, and so on.



The pouring of coffee is about to begin, with the keg of much-needed energy being poured into the mug at the top, allowing the excess of coffee to flow into the mugs below it until the keg runs out and all overflowed coffee has finished flowing into the infinite number of rows. You are curious to know how much coffee an arbitrary mug will have; however, you are too exhausted to do the calculations in your head, so you've deferred this task to your friend who had too much homework and was unable to attend the hackathon.

Input:

The first line of input will consist of two single space-separated integers, C and K , denoting the capacity of each cup, and the capacity of the full keg, respectively.

The second line of input will consist of two single space-separated integers, r and c denoting the queried row, and the queried mug from that row. Note that the top row of mugs begins at index 1 with each row below it going up by one, and the left-most mug on each row has index 1, with each mug to the right of it going up by one.

Constraints:

$$1 \leq C, K \leq 10^{10}$$

$$1 \leq r \leq 2.5 \cdot 10^3$$

$$1 \leq c \leq r$$

Output:

Output a single value f denoting the amount of liquid contained in the mug at row r and column c . Express this value with 5 decimal places of precision.

Sample input 1:

```
1 4
3 2
```

Sample output 1:

```
0.50000
```

Explanation 1:

The following is a visualization of how filled each mug is after all of the keg has been emptied out:

```

      1
    1  1
  0.25 0.5 0.25
```

(Additional Sample Input and Output on Next Page)

Computer Science Programming Packet – Code Bash - 2026

Sample input 2:

```
1 100000009
34 18
```

Sample output 2:

```
1.00000
```

Sample input 3:

```
3 20
4 2
```

Sample output 3:

```
1.25000
```

9. William

Program Name: William.java

Input File: william.dat

William is stuck in a maze. There are robots moving through the maze on tracks and he needs to avoid them to get out. You must write a program to determine the shortest path from his current position to the exit, without running into any robots as they move. You will be given a map of the maze, made up of the following characters:

- . – This character denotes an empty space in the maze, that will be passable in 1 second.
- # – This character denotes a wall, which is impassable.
- ? – This character denotes the starting point, where William is when it starts, passable in 1 second.
- @ – This character denotes the exit, passable in 1 second. Not guaranteed to exist.
- A-Z – This character denotes the starting position of a robot on its predetermined path.
- a-z – This character denotes the rest of the path, the robot will move one space every second, first trying to go up, then left, then down, then right (goes as far up as possible, then as far left as possible, then as far down as possible, then as far right as possible, taking one step at a time, should move in a circle). The robot will move AFTER William takes each step. The lowercase path corresponds to the uppercase robot (a -> A). Each path will form a cycle (a closed loop) that the robot will take if it checks directions in the order given (and I will not be mean with robot placement, it will always be on the top or right side of the cycle). This closed loop can be of any size.

William can only move in the 4 cardinal directions (up, down, left, right), and is allowed to stay in place, waiting for a robot to pass.

Input:

The input file will begin with an integer, n , denoting how many test cases there will be in the file. Each test case will begin with 3 space-separated integers, r , c , and o , denoting the number of rows and columns in the map, and the number of robots (between 0 and 26 inclusive), respectively. The following r lines will each contain c characters denoting the maze.

Output:

For each test case, output the minimum number of steps to escape the maze, or the string "Locked In." if it is not possible to escape the maze.

Sample input:

```
2
5 5 1
?....
.aaa.
.a@A.
.aaa.
.....
5 8 2
?...aaAa.
...a##a.
bBbaaaa.
b#b...##
bbb...@.
```

Sample output:

```
4
11
```

10. Xander

Program Name: Xander.java

Input File: xander.dat

Xander loves searching for words using wildcard patterns. In one of his patterns, a star (*) can stand for any sequence of characters – even no characters at all – and a question mark (?) stands for exactly one character. Every other character must match itself. Given a word and a pattern, Xander wants to know whether the pattern matches the **entire** word.

A pattern matches a word when it can be lined up so that:

- * matches any sequence of characters, including an empty sequence,
- ? matches exactly one character, and
- any other character matches only itself.

Input:

The first line of the input file contains an integer, *n*, the number of test cases. Each of the next *n* lines contains a word, a single space, and a pattern. The word is made of lowercase letters; the pattern is made of lowercase letters, *, and ?. Each word and each pattern is between 1 and 1000 characters long.

Output:

For each test case, output YES if the pattern matches the entire word, or NO if it does not.

Sample input:

```
6
codebash co*h
codebash *bash
codebash code
hello h?llo
abcde a**e
ab a?b
```

Sample output:

```
YES
YES
NO
YES
YES
NO
```

11. String Construction Pt. 1

Program Name: SC1.java

Input File: sc1.dat

While working on your most recent English assignment, you accidentally did a "replace all" removing every space in your word document. Unfortunately for you, `Ctrl + Z` isn't working as your laptop's `Ctrl` key is broken. For most people, this would take a long time to manually go back through the assignment and add spaces back where they should be, but you figure you have a better strategy than brute forcing adding the spaces back:

If you had a list containing at least all of the unique words that were contained within your English assignment, then you could run an algorithm that looks to see if a construction of those words could be equal to your "corrupt" string. If there does exist some construction, then you could run a follow-up algorithm that attempts to actually re-form the sentence.

For this part of the question, you will actually be writing the second of these two algorithms. You are guaranteed that there exists exactly one sequence of dictionary words that concatenates to form the corrupt string. Each dictionary word may be used zero, one, or multiple times.

Input:

The first line will consist of two single space-separated integers: a denoting the number of unique words in your dictionary, and b the length of the corrupt string. The second line will consist of a single space-separated strings of varying lengths, each contributing towards your dictionary. The third line will consist of a string of length b denoting the corrupt string you wish to analyze.

Constraints:

$$1 \leq a \leq 10^4$$

$$1 \leq b \leq 10^6$$

$$1 \leq |\text{word}_i| \leq 20$$

Each dictionary word consists of only lowercase English letters.

All dictionary words are unique.

Output:

Print a single line, denoting the repaired string. That is, the corrupt string, with spaces inserted between the words in the unique sequence that forms the corrupt string.

Sample input 1:

```
6 22
a are hat top rated tophats
tophatsareatopratedhat
```

Sample output 1:

```
tophats are a top rated hat
```

Explanation 1:

Each word is used exactly once. Note that the sentence could not read as "top hat s are a top rated hat" since the word "s" does not appear within the dictionary.

(Additional Sample Input and Output on Next Page)

Sample input 2:

```
11 22
a are hat top sadly rated tophats rubber duck the best
tophatsareatopratedhat
```

Sample output 2:

```
tophats are a top rated hat
```

Explanation 2:

This is identical to the first sample input; however, there are additional words contained in the dictionary that do not end up being used in the construction of the final sentence.

Sample input 3:

```
7 20
a an ant antenna ten tennis is
antennatenisanantten
```

Sample output 3:

```
antenna ten is an ant ten
```

12. String Construction Pt. 2

Program Name: SC2.java

Input File: sc2.dat

While working on your most recent English assignment, you accidentally did a "replace all" removing every space in your word document. Unfortunately for you, `Ctrl + Z` isn't working as your laptop's `Ctrl` key is broken. For most people, this would take a long time to manually go back through the assignment and add spaces back where they should be, but you figure you have a better strategy than brute forcing adding the spaces back:

If you had a list containing at least all of the unique words that were contained within your English assignment, then you could run an algorithm that looks to see if a construction of those words could be equal to your "corrupt" string. If there does exist some construction, then you could run a follow-up algorithm that attempts to actually re-form the sentence.

For this part of the question, you will be writing the first of these two algorithms with a slightly stronger constraint to prove. Namely, you wish to know the number of unique ways you can use the provided dictionary to construct the corrupt string (i.e., the number of unique constructions).

For this problem, a "construction" is a sequence of words from the provided dictionary whose concatenation is exactly equal to the corrupt string. Each dictionary word may be used zero, one, or multiple times.

Input

The first line will consist of two single space-separated integers: a denoting the number of unique words in your dictionary, and b the length of the corrupt string. The second line will consist of a single space-separated strings of varying lengths, each contributing towards your dictionary. The third line will consist of a string of length b denoting the corrupt string you wish to analyze.

Constraints

$$1 \leq a \leq 10^4$$

$$1 \leq b \leq 10^6$$

$$1 \leq |\text{word}_i| \leq 20$$

Each dictionary word consists of only lowercase English letters.

All dictionary words are unique.

Output

Print a single integer, denoting the number of unique permutations of the dictionary can construct the corrupt string. Since this number can be quite large, print this modulo $10^9 + 7$.

Sample input 1:

```
3 3
a aa aaa
aaa
```

Sample output 1:

```
4
```

Explanation 1:

There are four possible valid constructions here:

- a a a
- a aa
- aa a
- aaa

So, our answer is 4.

(Additional Sample Input and Output on Next Page)

Computer Science Programming Packet – Code Bash - 2026

Sample input 2:

```
8 8
a b c ab bc abc bca cba
abcbcaab
```

Sample output 2:

```
24
```

Explanation 2:

The following are *some* of the valid constructions:

- a b c b c a a b
- abc bca ab
- ab c bc a ab
- a b c b c a ab
- ...

It should be noted that since the "word" aa does not appear in the dictionary, we have to split the two consecutive a's among two separate words in the dictionary.

Sample input 3:

```
6 14
the cat sat on mat is
thecatsatonmat
```

Sample output 3:

```
1
```